Security Testing

CWE and OSWAP Top 10

(part 1)

Outline

- **CWE Top-10**
- OWASP Top-10 vulnerabilities (part 2)

Implementation and bugs: security perspective (1)

- **Bug**: the system isn't behaving as it's designed to behave
- **Vulnerability**: is a chain of weaknesses linked by causality: it starts with a bug and ends with a final error, which if exploited leads to a security failure.
  - A "weakness" is a condition in a software, firmware, hardware, or service component that, under certain circumstances, could contribute to the introduction of vulnerabilities.

- Many security issues are related to vulnerabilities due to bugs, defects, or design flaws in a targeted software system

Implementation and bugs: security perspective (2)

- To know typical (code) patterns that often lead to security issues
  - Not always! it depends on how they are used, the environment, etc.

- To know practical examples about Secure programming guidelines

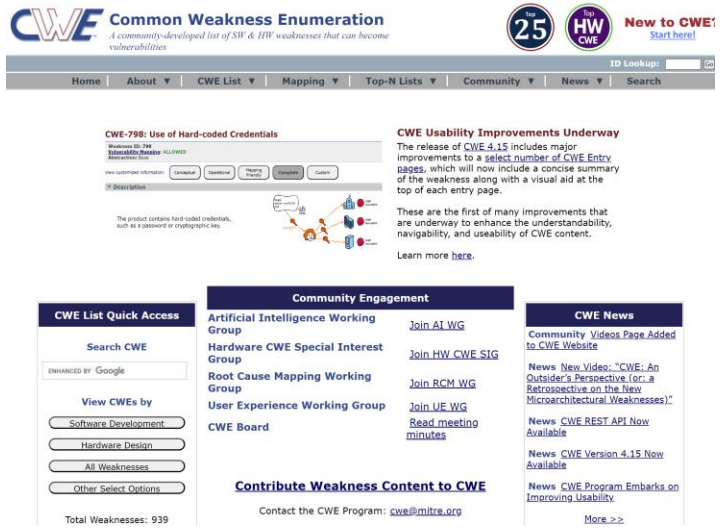
- To know what we have to search while analyzing and testing a system

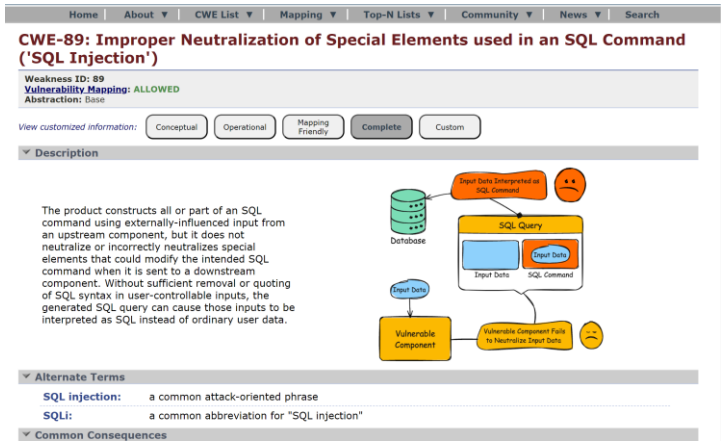
We are not doing penetration testing or playing as red teams (they aim at exploiting vulnerability and crashing the system), we aim at identifying any possible weaknesses that can lead to security issues

Common Weakness Enumeration (recall)

CWE (by MITRE)

- It provides a publicly available list of the most common software (and hardware) weaknesses that can contribute to security vulnerabilities.
- Each CWE is described by several fields

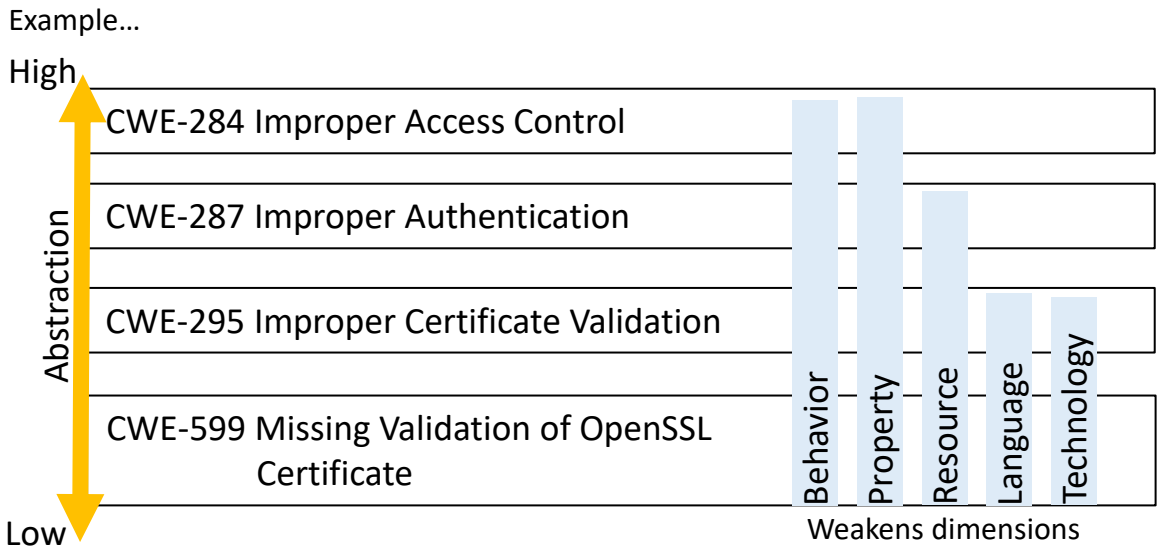


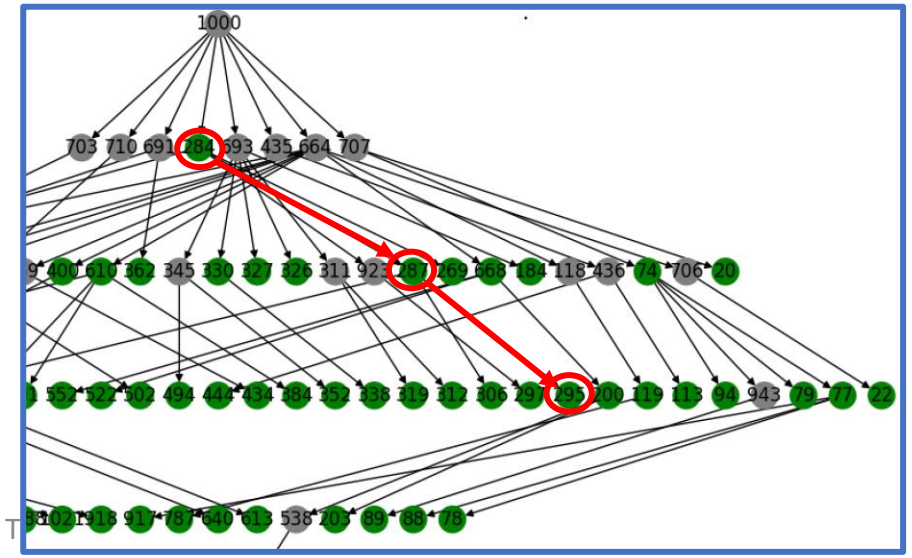


https://cwe.mitre.org
https://cwe.mitre.org/data/definitions/699.html

CWE: Common Weakness Enumeration

- It provides a list of the most common software (and hardware) weaknesses that can contribute to security vulnerabilities.
- Each CWE has a unique identifier
- Each CWE is described by several fields
- CWEs have different level of abstraction
- CWEs are hierarchically organized (“parent of”, “child of”, “can follow”, “member of”)

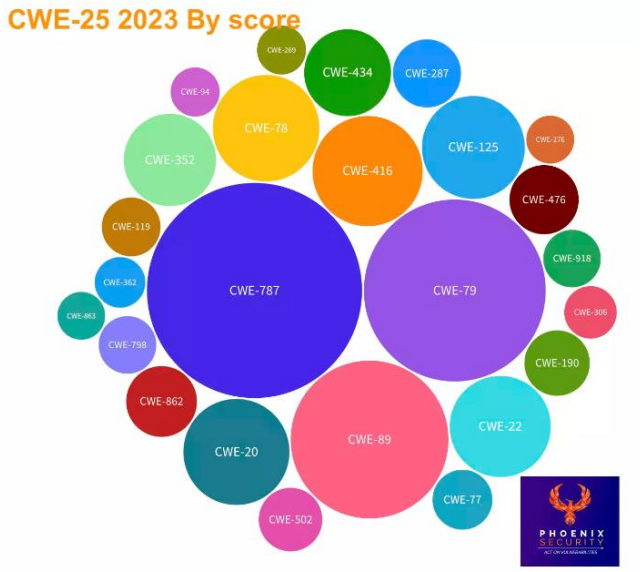




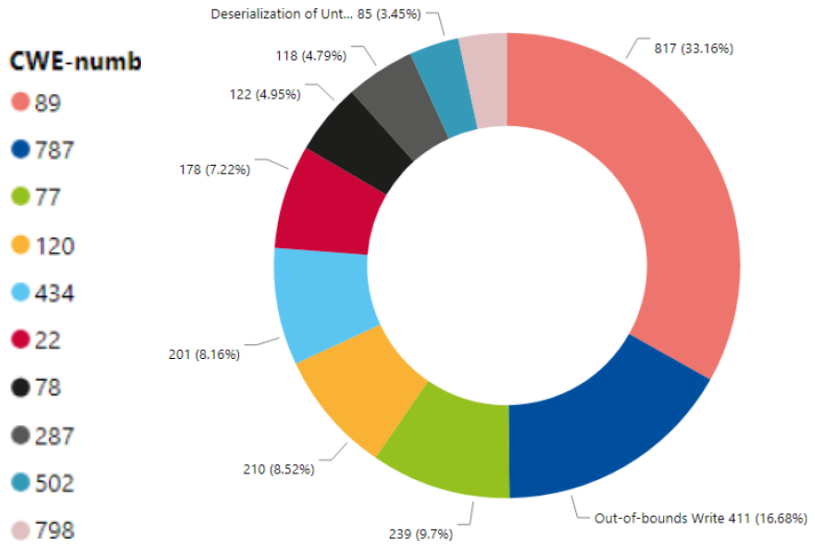
https://cwe.mitre.org

CWE: Top 10/25

- They are lists that includes the most dangerous software weaknesses across various applications and systems (not limited to a unique domain such as the web).
- They are compiled by scoring each CWE based on its prevalence (the number of times the CWE was mapped to a CVE) and (average) severity in real-world applications and the potential for damage if the weakness is exploited
- Top 10 vs Top 25
  - Top 10 is based on the known exploited vulnerabilities (by KEV)
  - differences: source of information, how CWEs are easy to be found by code scanning tools, how CWEs are easy to exploit, what is the impact of the CWEs for adversaries that exploit them



CWE by critical severity CVEs (nr. and %) [ENISA 2023]

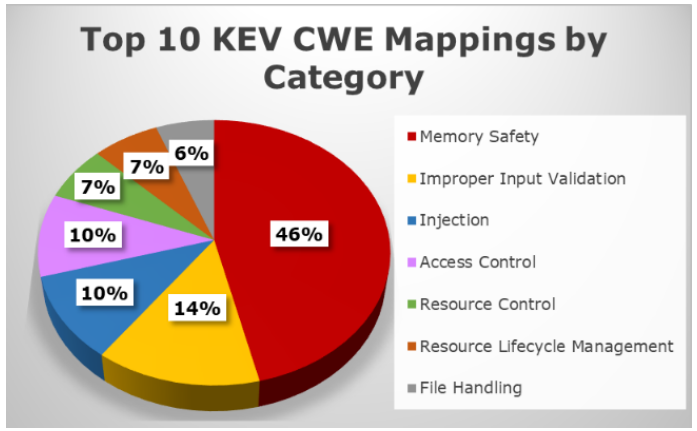


From the analysis of the known exploited vulnerabilities (KEV)
https://cwe.mitre.org/top25/archive/2023/2023_kev_list.html

CWE Top-10 Weaknesses

1. Use-After-Free (CWE-416)
2. Heap-based Buffer Overflow (CWE-122)
3. Out-of-bounds Write (CWE-787)
4. Improper Input Validation (CWE-20)
5. Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection') (CWE-78)
6. Deserialization of Untrusted Data (CWE-502)
7. Server-Side Request Forgery (SSRF) (CWE-918)
8. Access of Resource Using Incompatible Type ('Type Confusion') (CWE-843)
9. Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal') (CWE-22)
10. Missing Authentication for Critical Function (CWE-306)

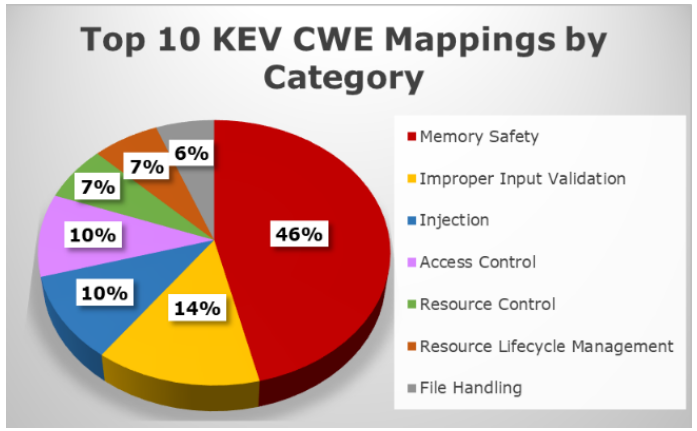
From the analysis of the known exploited vulnerabilities (KEV)
https://cwe.mitre.org/top25/archive/2023/2023_kev_list.html



CWE Top-10 Weaknesses

1. Use-After-Free (CWE-416)
2. Heap-based Buffer Overflow (CWE-122)
3. Out-of-bounds Write (CWE-787)
4. Improper Input Validation (CWE-20)
5. Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection') (CWE-78)
6. Deserialization of Untrusted Data (CWE-502)
7. Server-Side Request Forgery (SSRF) (CWE-918)
8. Access of Resource Using Incompatible Type ('Type Confusion') (CWE-843)
9. Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal') (CWE-22)
10. Missing Authentication for Critical Function (CWE-306)

From the analysis of the known exploited vulnerabilities (KEV)
https://cwe.mitre.org/top25/archive/2023/2023_kev_list.html



Use-After-Free (CWE-416)

Description:

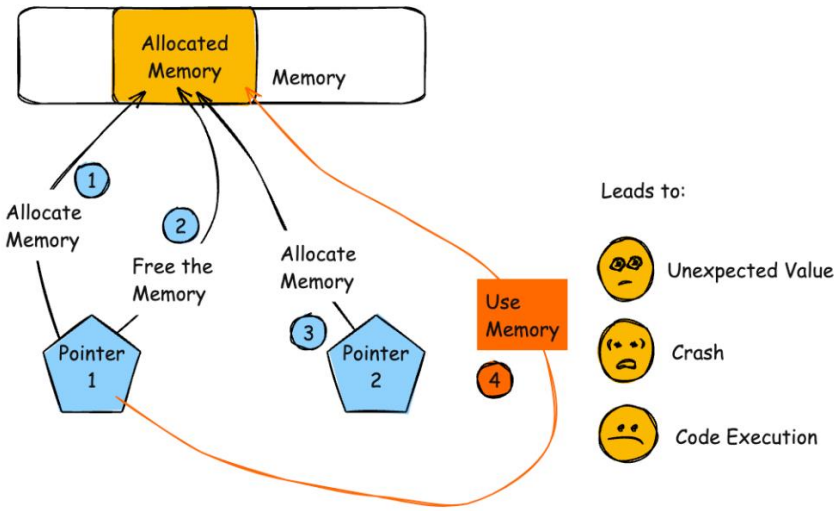
- The product reuses or references memory after it has been freed.
- At some point afterward, the memory may be allocated again and saved in another pointer, while the original pointer references a location somewhere within the new allocation.
- Any operations using the original pointer are no longer valid because the memory "belongs" to the code that operates on the new pointer.

Prevention:

- When freeing pointers, be sure to set them to NULL once they are freed.
- Choose a language that provides automatic memory management.

Detection

- Fuzzing
- Automatic static analysis



Related attacks:

- Code Injection (CAPEC-242)
- Overflow Buffers (CAPEC-100)

https://cwe.mitre.org/data/definitions/416.html

Use-After-Free (CWE-416): example

```c
char* ptr = (char*)malloc (SIZE);
if (err) {
  abrt = 1;
  free(ptr);
}
...
if (abrt) {
   logError("operation aborted before commit", ptr);
}
```

C

*void *malloc(size);*
Allocates size bytes of uninitialized storage. If allocation succeeds, returns a pointer. This memory must be explicitly freed in order to avoid memory leaks

void free(memblock);
De-allocates the *memblock* previously allocated with a malloc

Example:

- When an error occurs, the pointer is immediately freed.
- However, this pointer is later **incorrectly used** in the *logError* function.

Use-After-Free (CWE-416): note

```java
public class UseAfterFree_Example {
   public static void main(String[] args) {
      FileInputStream fis = null;
      try {
         fis = new FileInputStream("example.txt"); // Open the file input stream...
         // Use the file input stream...
      } catch (IOException e) {
         e.printStackTrace(); // Print the log if the exception is raised
      } finally {
         // Executed at the end of the block execution (with/without exceptions)
         if (fis != null) {
            try {
               fis.close(); // Close of  the file input stream...
            } catch (IOException e) {
               e.printStackTrace();
            }
         }
      }
      try {
         int data = fis.read();  // Use of the file input stream, but it has been closed
      } catch (IOException e) {
         System.out.println("Attempted to read from a closed stream.");
      }
   }
}
```

try{ …block1… } catch (exception) { …block2… } finally { …block3…}

- *block1* is executed
- an *exception* is raised if an issue occurred during the execution of *block1 and*, in such a case, the *exception* can be captured by the catch clause and *block2* executed
- *block3* is executed when *block1* is executed without issues, or when *block2* is executed after an issue during the execution of *block1*

- UAF not only relates to pointers and memory.
- UAF(-like) weaknesses can be found also when other resources (e.g., **file handles or network connections**) are mismanaged.
- Hence, UAF can occur also in languages (e.g., Java) with automatic memory management system.

https://svenruppert.com/2024/05/17/cwe-416-use-after-free-vulnerabilities-in-java/

Use-After-Free (CWE-416): CVEs

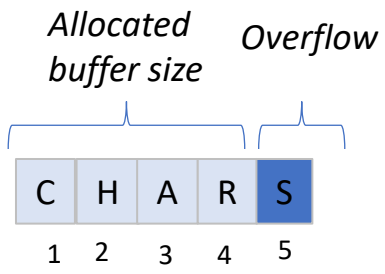
Examples of CVEs:

- CVE-2022-20141: Android system kernel has insufficient resource locking (CWE-413) leading to UAF
  https://www.cve.org/CVERecord?id=CVE-2022-20141
- CVE-2022-2621: Two threads in a web browser use the same resource (CWE-366), but one of those threads can destroy the resource before the other has completed
  https://www.cve.org/CVERecord?id=CVE-2022-2621
- CVE-2020-6819: Race condition (CWE-362) lades to UAF
  https://www.cve.org/CVERecord?id=CVE-2020-6819
- CVE-2010-4168: Use-after-free triggered by closing a connection while data is still being transmitted.
  https://www.cve.org/CVERecord?id=CVE-2010-4168

Heap-based Buffer Overflow (CWE-122)

Description:

- A heap overflow condition is a buffer overflow, where the buffer that can be overwritten is allocated in the heap portion of memory, generally meaning that the buffer was allocated using malloc() or new().



Prevention:

- Use automatic buffer overflow detection mechanisms
- Implement and perform bounds checking on input.
- Do not use dangerous functions such as gets. Look for their safe equivalent, which checks for the boundary.

Detection:

- Fuzzing
- Automated static analysis

Related attacks:

- Forced Integer Overflow (CAPEC-92)
- Overflow Buffers (CAPEC-100)

https://cwe.mitre.org/data/definitions/122.html

Heap-based Buffer Overflow (CWE-122): example

```c
#define BUFSIZE 4
int main(int argc, char **argv) {
  char *buf;
  buf = (char *)malloc(sizeof(char)*BUFSIZE);
  strcpy(buf, argv[1]);
}
```

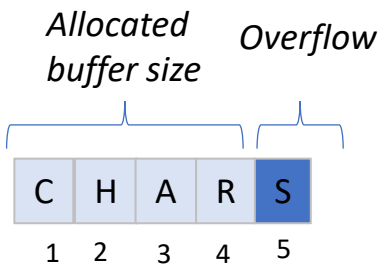
C

*void *malloc(size_t size);*
Allocates size bytes of uninitialized storage. If allocation succeeds, returns a pointer

*strcpy(*destination, *source)*
copies from destination to source, till the end NULL character ('\0')

Example

- The buffer is allocated heap memory with a fixed size, but there is no guarantee the string in argv[1] will not **exceed this size** and cause an overflow.



Heap-based Buffer Overflow (CWE-122): CVEs

Examples of CVEs:

- CVE-2021-29529: a machine-learning product can have a heap-based buffer overflow (CWE-122) when some integer-oriented bounds are calculated by using ceiling() and floor() on floating point values (CWE-1339)
  https://www.cve.org/CVERecord?id=CVE-2021-29529
- CVE-2021-43537: in a web browser, an unsigned 64-bit integer is forcibly cast to a 32-bit integer (CWE-681) and potentially leading to an integer overflow (CWE-190). If an integer overflow occurs, this can cause heap memory corruption (CWE-122)
  https://www.cve.org/CVERecord?id=CVE-2021-43537
- CVE-2007-4268: integer signedness error (CWE-195) passes signed comparison, leading to heap overflow
  https://www.cve.org/CVERecord?id=CVE-2007-4268
- CVE-2009-2523: product does not handle when an input string is not NULL terminated (CWE-170), leading to buffer over-read or heap-based buffer overflow
  https://www.cve.org/CVERecord?id=CVE-2009-2523

Out-of-bounds Write (CWE-787)

Description:

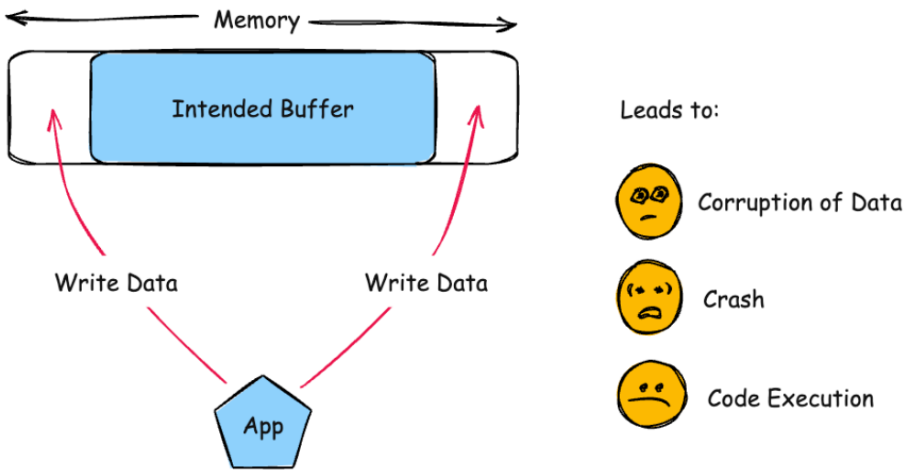
- The product writes data past the end, or before the beginning, of the intended buffer.
- Often used to describe the consequences of writing to memory outside the bounds of a buffer, or to memory that is otherwise invalid.

Prevention:

- Double check that the buffer is as large as specified and check buffer boundaries.
- Replace unbounded copy functions with analogous functions that support length arguments, such as strcpy with strncpy. Create these if they are not available.

Detection:

- Automated static analysis
- Automated dynamic analysis (fuzzing, fault-injection based testing)



Note: buffer overflow is a form of out-of-bounds write that could happen for buffers (e.g., arrays)
→ Parent of CWE-122

Related attacks:

- Overflow Buffers (CAPEC-100)
- Code Injection (CAPEC-242)

https://cwe.mitre.org/data/definitions/787.html

Out-of-bounds Write (CWE-787): examples

```c
int id_sequence[3];

id_sequence[0] = 123;
id_sequence[1] = 234;
id_sequence[2] = 345;
id_sequence[3] = 456;
```

Example
- The code saves four numbers into an array
- But the array is allocated to hold three elements, thus 0-2 is the range of valid indices (no compile-time error)
- The assignment to *id_sequence*[3] is **out of bounds**

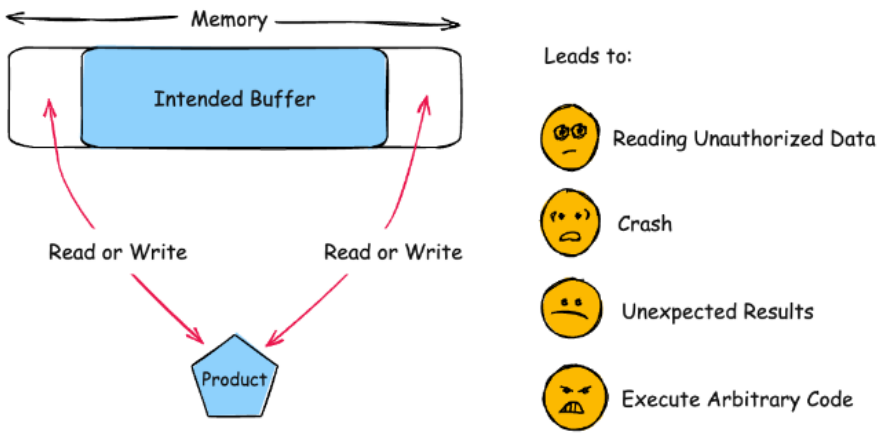
```c
int main() {
 ...
 char *result = strstr(destBuf, "Replace Me");
 int idx = result - destBuf;
 strcpy(&destBuf[idx], srcBuf);
 ...
}
```

*strstr(*string, *substring):* returns a pointer to the found *substring* in *string* or NULL if there is no match
*strcpy(*destination, *source):* copies from destination to source
&var: gives the address of var

Example
- The code replaces the substring "Replace Me" in *destBuf* with the string in *srcBuf*.
- If the substring is not found in destBuf, strstr() will return ***NULL***, causing the pointer *idx* to be undefined, potentially setting the value of *idx* to a negative number.
- A negative *idx* results in a buffer underwrite of *destBuf*

Out-of-bounds Write (CWE-787): note

CWE-119: Improper Restriction of Operations within the Bounds of a Memory Buffer



Description:

- The product performs operations on a memory buffer, but it **reads from or writes to a memory location outside the buffer's intended boundary**.
- This may result in <u>read or write operations on unexpected memory locations</u> that could be linked to other variables, data structures, or internal program data.

CWE -119 → CWE -787 → CWE -122

Note:
→ CWE-119 is Parent of CWE-787

Related attacks:
- Overflow Buffers (CAPEC-100)
- Client-side Injection-induced Buffer Overflow (CAPEC-14)
- Buffer Overflow in an API Call (CAPEC-8)

https://cwe.mitre.org/data/definitions/119html

Out-of-bounds Write (CWE-787): CVEs

Examples of CVEs:

- CVE-2021-21220: insufficient input validation (CWE-20) in browser allows heap corruption
  https://www.cve.org/CVERecord?id=CVE-2021-21220
- CVE-2020-17087: integer truncation (CWE-197) causes small buffer allocation leading to out-of-bounds write
  https://www.cve.org/CVERecord?id=CVE-2020-17087
- CVE-2019-1010006: compiler optimization (CWE-733) removes or modifies code used to detect integer overflow
  https://www.cve.org/CVERecord?id=CVE-2019-1010006
- CVE-2020-0968: A remote code execution vulnerability exists in the way that the scripting engine handles objects in memory in Internet Explorer, aka 'Scripting Engine Memory Corruption Vulnerability'
  https://www.cve.org/CVERecord?id=CVE-2020-0968

Improper Input Validation (CWE-20)

High-level

Description:

- The product receives input values, but it **does not validate or incorrectly validates** them.
- Input validation is a frequently-used technique for checking potentially dangerous inputs in order to ensure that the inputs are safe for processing within the code
- In the case of no-validation, unintended input values can alter the program control flow.

Prevention:

- Use an input validation framework, e.g., Struts, OWASP ESAPI Validation APIs
- Check all the potentially untrusted inputs (e.g., parameters, arguments, cookies, anything read from the network)
- Assume all input is malicious.

Detection:

- Automated/Manual static analysis
- Automated dynamic analysis (fuzzing, fault-injection testing)

Related attacks:

- Command Line Execution through SQL Injection (CAPEC-108)
- Format String Injection (CAPEC-135)
- Input Data Manipulation (CAPEC-153)
- Blind SQL Injection (CAPEC-7)
- Server-side Request Forgery (CAPEC-664)
- Cross-Site Scripting (XSS) (CAPEC-63)

Improper Input Validation (CWE-20): examples

```java
...
public static final double price = 20.00;
int quantity = currentUser.getAttribute("quantity");
double total = price * quantity;
chargeUser(total);
...
```

Example

- The code shows that a user can specify the **quantity** of items to be purchased, and a total is calculated.
- The quantity can be a negative value.
- A negative value of quantity can lead to an account credited instead of debit.

```java
private void buildList ( int untrustedListSize ){
  if ( 0 > untrustedListSize ){
   die("Negative value supplied for list size!");
  }
  Widget[] list = new Widget [ untrustedListSize ];
  list[0] = new Widget();
}
```

Example

- This code takes a user-supplied value to allocate an array of objects.
- This example checks to ensure that a non-negative value is supplied.
- However, if 0 is provided, the code builds an array of size 0 and aim at storing a new Widget in the first item of the list, causing an issue.

Improper Input Validation (CWE-20): CVEs

Examples of CVEs:

- CVE-2024-37032: Large language model (LLM) management tool does not validate the format of a digest value (CWE-1287) from a private, untrusted model registry, enabling relative path traversal
  https://www.cve.org/CVERecord?id=CVE-2024-37032
- CVE-2020-9054: improper input validation (CWE-20) in username parameter, leading to OS command injection
  https://www.cve.org/CVERecord?id=CVE-2020-9054
- CVE-2021-30860: improper input validation (CWE-20) leads to integer overflow (CWE-190) in mobile OS
  https://www.cve.org/CVERecord?id=CVE-2021-30860
- CVE-2021-22205: backslash followed by a newline can bypass a validation step, leading to eval injection
  https://www.cve.org/CVERecord?id=CVE-2021-22205

Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection') (CWE-78)

Description:

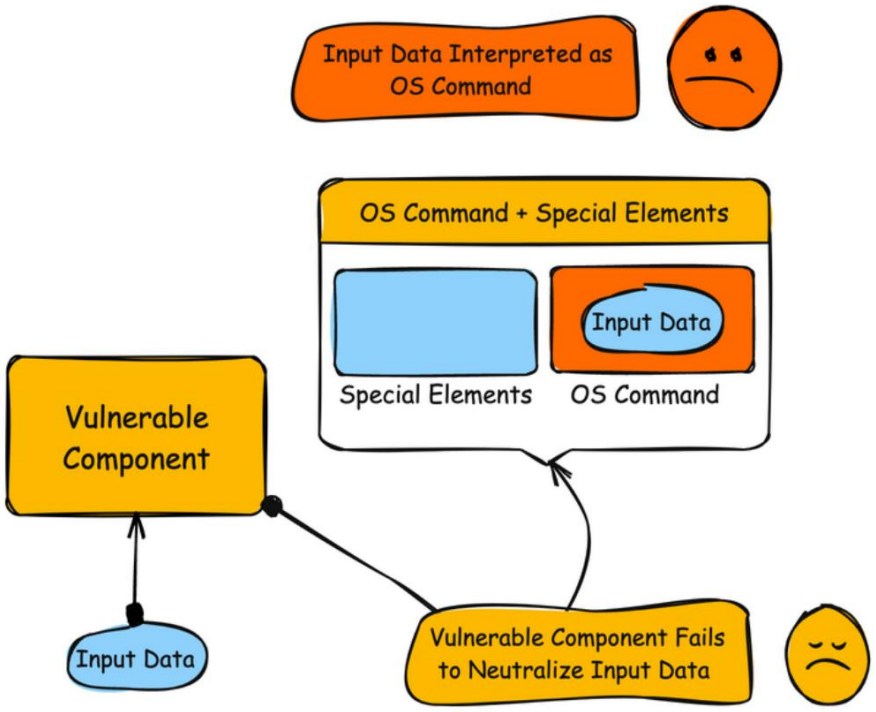
- The product constructs an **OS command using externally-influenced input**, but it does not neutralize (or it incorrectly neutralizes) special elements that could modify the intended OS command.

Prevention:

- Use library calls rather than external processes to recreate the desired functionality.
- For any security checks performed on the client side, ensure that these checks are duplicated on the server side
- If the program to be executed allows arguments to be specified within an input file or from standard input, then consider using that mode to pass arguments instead of the command line.

Detection:

- Automated static analysis (not perfect solution since APIs cannot be analyzed/considered by tools)
- Automated dynamic analysis (fuzzing, fault-injection based testing



Related attacks:

- OS Command Injection (CAPEC-88)
- Command Line Execution through SQL Injection (CAPEC-108)

https://cwe.mitre.org/data/definitions/78.html

Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection') (CWE-78): example

```php
$userName = $_POST["user"];
$command = 'ls -l /home/' . $userName;
system($command);
```

system(OS_command): it is used to invoke an operating system command from a program

Example

- The code takes the name of a user and lists the contents of that user's home directory.
- *$userName* is not checked for malicious input.
- *$userName* can **contain an arbitrary OS command** such as:
  "; rm -rf /"
- Which would result in *$command* being:
  "ls -l /home/;rm -rf /"
- Since the semi-colon is a command separator in Unix, the OS would first execute the *ls* command, then the *rm* command, deleting the entire file system.

Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection') (CWE-78): CVEs

Examples of CVEs:

- CVE-2020-10987: OS command injection in Wi-Fi router, it allows remote attackers to execute arbitrary system commands via the deviceName POST parameter.
  https://www.cve.org/CVERecord?id=CVE-2020-10987
- CVE-2020-9054: improper input validation (CWE-20) in username parameter, leading to OS command injection
  https://www.cve.org/CVERecord?id=CVE-2020-9054
- CVE-1999-0067: CGI program does not neutralize "|" metacharacter when invoking a phonebook program.
  https://www.cve.org/CVERecord?id=CVE-1999-0067
- CVE-2003-0041: FTP client does not filter "|" from filenames returned by the server, allowing for OS command injection.
  https://www.cve.org/CVERecord?id=CVE-2003-0041

Deserialization of Untrusted Data (CWE-502)

Description:

- It is often convenient to **serialize objects** for communication or to save them for later use.
- The product **deserializes untrusted data** without sufficiently verifying that the resulting data will be valid.
- Deserialized data or code can be modified without using the provided accessor functions if it does not use cryptography to protect itself.

Prevention:

- When deserializing data, populate a new object rather than just deserializing.
- Ensure that the integrity is not compromised by using a digital signature (HMAC) to the serialized data that is validated before deserialization

Detection:

- Automated static analysis

Related attacks:

- Object injection (CAPEC-586)

https://cwe.mitre.org/data/definitions/502.html

Deserialization of Untrusted Data (CWE-502): examples

```java
try {
 File file = new File("object.obj");
 ObjectInputStream in = new ObjectInputStream(new FileInputStream(file));
 javax.swing.JButton button = (javax.swing.JButton) in.readObject();
 in.close();
}
```

Example

- The code deserializes an object from a file and uses it as a UI button
- This code **does not verify the source or contents** of the file before deserializing it
- An attacker may be able to replace the intended file with a file that contains arbitrary malicious code which will be executed when the button is pressed.

Deserialization of Untrusted Data (CWE-502): CVEs

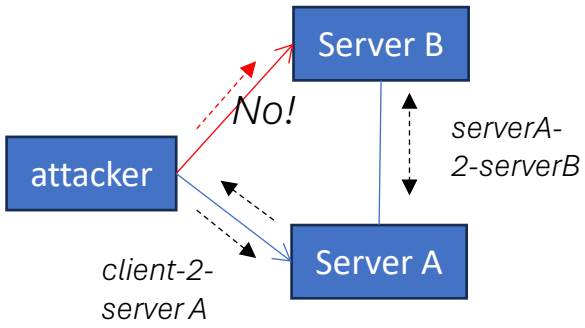
Examples of CVEs:

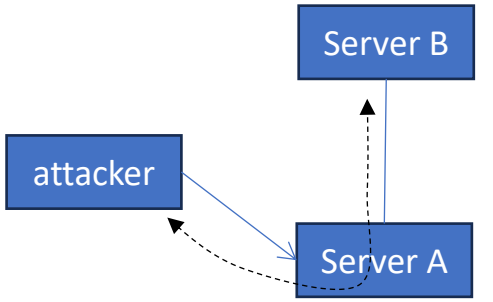
- CVE-2019-12799: bypass of untrusted deserialization issue (CWE-502) by using an assumed-trusted class
  https://www.cve.org/CVERecord?id=CVE-2019-12799
- CVE-2015-8103: Deserialization issue in commonly-used Java library allows remote execution
  https://www.cve.org/CVERecord?id=CVE-2015-8103
- CVE-2013-1465: use of PHP unserialize function on untrusted input allows attacker to modify application configuration.
  https://www.cve.org/CVERecord?id=CVE-2013-1465

Server-Side Request Forgery (SSRF) (CWE-918)

Description:

- Context: a server-to-server or service-to-service trust relationships exist
- Issue: a security vulnerability in a server allows an attacker to cause the server-side application to make requests to an unintended location, e.g., another server or service
  - The attacker sends requests from the back end of a server to another server or to a local service.
  - The server or service that receives that request believes that the request came from the application and is legitimate
  - The attacker might cause the server/service to make a connection to reserved servers/services
- The attackers can bypass the access controls (e.g., firewalls) that prevent the direct attack to the target server, by means to the truest relation between servers.
- The server can be used as a proxy to conduct port scanning of hosts in internal networks, use other URLs such as that can access documents on the system





client-2-serverB via serverA

Prevention:

- Limit outbound traffic from the application server
- Build a whitelist of trusted domains and IP addresses
- Sanitizing user input to prevent potential risks

Detection:

- Automated static analysis

Related attacks:

- Server-side Request Forgery (CAPEC-664)

https://cwe.mitre.org/data/definitions/918.html

Server-Side Request Forgery (SSRF) (CWE-918): example

```php
$url = $_GET['url’];   // e.g., http://localhost/?url=http://test/images/logo.gif
$image = fopen($url, 'rb’);      //send a request to get the image/file/resource
header("Content-Type: image/png");    //send the response headers
fpassthru($image);     //get the image
```

fopen(url, mode): it opens a file or an URL, according to the indicated mode (e.g., *r* for reading, etc.)

header(*header_string*): it sends a raw HTTP *header_string* to a client

fpassthru(*file*): it reads from the current position in a *file*

Example

- The code get the value of the *url* parameter, and then it load a file (logo.gif)
- However, there is no control of the *url*, so an attacker can change it and for instance the following requests, thus accessing to other resources of the server:

GET /?url=http://localhost/server-status HTTP/1.1

GET /?url=file:///etc/passwd HTTP/1.1

Server-Side Request Forgery (SSRF) (CWE-918): CVEs

Examples of CVEs:

- CVE-2021-26855: Server Side Request Forgery (SSRF) in email server
  https://www.cve.org/CVERecord?id=CVE-2021-26855
- CVE-2021-21973: Server Side Request Forgery in cloud platform, due to an improper URLs validation
  https://www.cve.org/CVERecord?id=CVE-2021-21973
- CVE-2016-4029: Incorrect validation of intended decimal-based IP address format (CWE-1286) enables parsing of octal or hexadecimal formats (CWE-1389), allowing bypass of an SSRF protection mechanism
  https://www.cve.org/CVERecord?id=CVE-2016-4029
- CVE-2002-1484: Web server allows attackers to request a URL from another server, including other ports, which allows proxied scanning.
  https://www.cve.org/CVERecord?id=CVE-2002-1484

Access of Resource Using Incompatible Type ('Type Confusion') (CWE-843)

Description:

- The product allocates or initializes a resource such as a pointer, object, or variable using one type, but it later **accesses that resource using a type that is incompatible** with the original type.
- When the product accesses the resource using an incompatible type, this could trigger logical errors because the resource does not have expected properties.
- Type confusion can lead to out-of-bounds memory access.

Prevention:

- Attention to (implicit and explicit) type conversion
- Check type before using specific properties

Detection:

- Static type analysis

Related attacks:

- Exploiting Trust in Client (CAPEC-22)

https://cwe.mitre.org/data/definitions/846.html

Access of Resource Using Incompatible Type ('Type Confusion') (CWE-843): examples

```php
1. $value = $_GET['value'];
2. $sum = $value + 5;
3. echo "value parameter is '$value'<p>";
4. echo "SUM is $sum";
```

http://link/page.php?value1=1&value2=2
Then $_GET[‘value1’] and $_GET[‘value2’] to get the value 1 and 2

http://link/page.php?value[]=1&value[]=2&value[]=3
Then $_GET[‘value'] will be an array of 1,2,3 values.

Example

- The PHP code accepts a value, adds 5, and prints the sum.

- When called with the following query string:
  value=123
- the program calculates the sum and prints out:
  SUM is 128
- However, the attacker could supply a query string such as:
  value[]=123
- The "[]" array syntax causes *$value* to be treated as an array type, which then generates a fatal error when calculating *$sum*:
  Fatal error: Unsupported operand types in program.php on line 2

Access of Resource Using Incompatible Type ('Type Confusion') (CWE-843): CVEs

Examples of CVEs:

- CVE-2010-4577: Type confusion in CSS sequence leads to out-of-bounds read.
  https://www.cve.org/CVERecord?id=CVE-2010-4577
- CVE-2011-0611: Size inconsistency in a "group of included constants," object type confusion allows code execution, first discovered when it was actively exploited in-the-wild.
  https://www.cve.org/CVERecord?id=CVE-2011-0611
- CVE-2010-0258: improperly-parsed file containing records of different types leads to code execution when a memory location is interpreted as a different object than intended.
  https://www.cve.org/CVERecord?id=CVE-2010-0258

Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal') (CWE-22)

Description:

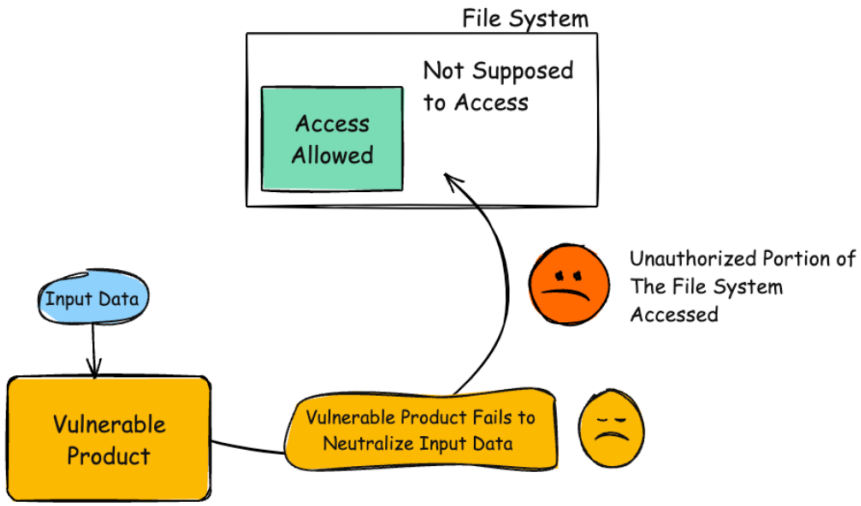
- The product **uses external input to construct a pathname** for a file or directory, located into a (restricted) parent directory
- Without a proper neutralization of special elements within the pathname, the path can relate to a different location (not supposed to be accessible).
- For instance, by separators such as ".." and "/", attackers can escape outside of the restricted location to access files or directories that are elsewhere on the system.

Prevention:

- Assume all input is malicious
- Store library, include, and utility files outside of the web document root, if possible.
- When the set of acceptable objects, such as filenames or URLs, is limited or known, create a mapping from a set of fixed input values (such as numeric IDs) to the actual filenames or URLs, and reject all other inputs.

Detection:

- Automatic/Manual static analysis
- Automatic dynamic analysis (vulnerability scanning)



Related attacks:

- Path traversal (CAPEC-126)
- Exploiting Trust in Client (CAPEC-22)

https://cwe.mitre.org/data/definitions/22.html

Access of Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal') (CWE-22): examples

```java
String filename = System.getProperty("com.domain.application.dictionaryFile");
File dictionaryFile = new File(filename);
```

Example

- The path to a dictionary file is read from a system property (e.g., environmental variable) and used to initialize a File object.
- However, the pathname of the file is not validated adequately before creating the File object.
- This allows anyone **who can control the system property** to determine what file is used.
- (Ideally, the path should be resolved relative to some kind of application or user home directory.)

```java
String path = getInputPath();
if (path.startsWith("/safe_dir/"))
{
File f = new File(path);
f.delete()
}
```

Example

- The code **attempts to validate a given input path** by checking it against an allowlist and once validated delete the given file. In this specific case, the path is considered valid if it starts with the string "/safe_dir/".
- However, an attacker could provide an input such as this: “/safe_dir/../important.dat”
- The software assumes that the path is valid because it starts with the "/safe_path/" sequence, but the "../" sequence will cause the program to delete the important.dat file in the parent directory

Access of Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal') (CWE-22): CVEs

Examples of CVEs:

- CVE-2024-4315: API for text generation using Large Language Models (LLMs) does not include the "\" Windows folder separator in its denylist (CWE-184) when attempting to prevent Local File Inclusion via path traversal (CWE-22), allowing deletion of arbitrary files on Windows systems.
  https://www.cve.org/CVERecord?id=CVE-2024-4315
- CVE-2019-20916: Python package manager does not correctly restrict the filename specified in a Content-Disposition header, allowing arbitrary file read using path traversal sequences such as "../“
  https://www.cve.org/CVERecord?id=CVE-2019-20916
- CVE-2022-31503: Python package constructs filenames using an unsafe os.path.join call on untrusted input, allowing absolute path traversal because os.path.join resets the pathname to an absolute path that is specified as part of the input.
  https://www.cve.org/CVERecord?id=CVE-2022-31503

Missing Authentication for Critical Function (CWE-306)

Description:

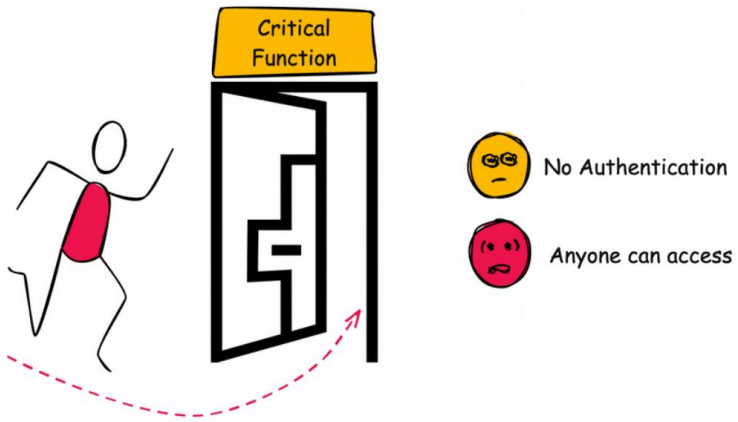
- The product **does not perform any authentication** for functionality that requires a provable user identity or consumes a significant amount of resources.
- Exposing critical functionality essentially provides an attacker with the privilege level of that functionality.

Prevention:

- For any security checks that are performed on the client side, ensure that these checks are duplicated on the server side
- Where possible, avoid implementing custom authentication routines and consider using authentication capabilities as provided by libraries, frameworks, operating systems, etc.

Detection:

- Automatic/Manual static analysis
- Automatic dynamic analysis (vulnerability scanning)



Related attacks:

- Cross Site Request Forgery (CAPEC-62)
- Communication Channel Manipulation (CAPEC-216)

https://cwe.mitre.org/data/definitions/306.html

Missing Authentication for Critical Function (CWE-306): CVEs

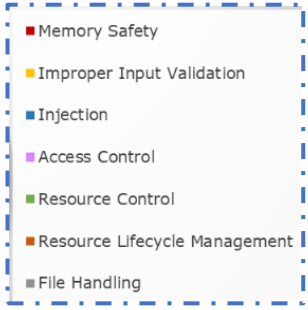
Examples of CVEs:

- CVE-2022-31260: a digital asset management program has an undisclosed backdoor in the legacy version of a PHP script (CWE-912) that could allow an unauthenticated user to export metadata
  https://www.cve.org/CVERecord?id=CVE-2022-31260
- CVE-2022-29951: TCP-based protocol in Programmable Logic Controller (PLC) has no authentication.
  https://www.cve.org/CVERecord?id=CVE-2022-29951
- CVE-2022-29952: Condition Monitor firmware uses a protocol that does not require authentication.
  https://www.cve.org/CVERecord?id=CVE-2022-29952
- CVE-2022-30317: Distributed Control System (DCS) uses a protocol that has no authentication.
  https://www.cve.org/CVERecord?id=CVE-2022-30317

CWE Top-10 …but not only

E.g., among the Top-25 most dangerous CWEs we can (additionally) find:

- CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
- CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
- CWE-352: Cross-Site Request Forgery (CSRF)
- CWE-476: NULL Pointer Dereference
- CWE-190: Integer Overflow or Wraparound
- CWE-362: Concurrent Execution using Shared Resource with Improper Synchronization ('Race Condition')
- CWE-94: Improper Control of Generation of Code ('Code Injection')
- ..



https://cwe.mitre.org/top25/archive/2023/2023_top25_list.html
CWE page: https://cwe.mitre.org/data/definitions/???.html
E.g., https://cwe.mitre.org/data/definitions/89.html